Fail-Closed Research Execution: Receipt Semantics and Independence Contracts

Anonymous Authors

Abstract

Tool-using research systems can produce an answer and a detailed execution trace while leaving unresolved who executed the work, which evidence was available, whether two agreeing runs were independent, and whether an infrastructure failure was accidentally admitted as scientific evidence. We define a fail-closed execution contract that separates request identity, result integrity, executor attribution, evidence admission and scientific authority. The framework contributes a typed receipt model, an explicit six-level independence taxonomy and an adversarial protocol for testing identity substitution, evidence replay, cross-lane leakage, partial writes and false success. Two contract properties follow from the definitions: capability failures cannot enter the scientific-evidence set when admission is type checked, and an agreement label cannot exceed the weakest verified independence predicate. No protected reliability result is reported. Unit tests and deterministic source scans, where available, establish only implementation conformance to their checked cases. The framework therefore makes execution claims more attributable and falsifiable without treating provenance as evidence quality or treating agreement as scientific corroboration.

1 Scope and nonclaims

Computational research increasingly depends on workflows that combine programs, data services, models, tools and human decisions. Established provenance models describe the entities, activities and agents involved in producing an object [6]; workflow profiles and research-object packages make such records more interoperable and reusable [4, 5, 10]. These developments make an execution easier to inspect, but they do not by themselves answer three distinct questions: whether the recorded executor identity is authentic, whether an output is scientifically admissible, and whether agreeing outputs constitute independent corroboration.

This paper addresses that boundary. Its object is a contract between an execution request, a result receipt and an evidence-admission decision. The contract is designed for research systems in which a host may call local processes, remote services, language models or other bounded capabilities. It is not a new workflow language, provenance ontology, cryptographic signature scheme or research agent. It instead specifies the minimum predicates that must be checked before a receipt can support progressively stronger statements about execution and independence.

The paper makes three contributions. First, it separates the identity of a request from the integrity, attribution and scientific status of its result. Second, it defines six agreement levels, from deterministic structural agreement to scientific corroboration, and makes unsupported upward relabeling invalid. Third, it provides a prospective hostile protocol whose attacks can falsify each guarantee.

The claim boundary is strict. A receipt can show that named bytes were processed under a declared configuration; it cannot establish that the input evidence was true or sufficient. A passing unit suite can show implementation conformance; it cannot estimate reliability outside the checked cases. Two requests can agree while sharing a model, executor or evidence corpus; agreement alone is not independence. The framework grants no scientific truth, novelty, publication, deployment or adoption authority.

2 Threat model and trust roots

2.1 Objects and principals

Let a campaign contain requests $q \in Q$, execution attempts $x \in X$, receipts $r \in R$, evidence objects $e \in E$ and adjudications $a \in A$. An executor principal is an authenticated subject allowed to perform an attempt. A provider is the service or runtime that performs a model-mediated operation, and a checkpoint is the immutable model identity when such an operation is used. A verifier checks receipt structure and cryptographic or content bindings. An adjudicator applies a frozen scientific protocol to admissible evidence. These roles may be implemented by one organization, but their logical separation must remain explicit.

The protected objects are not only final files. They include the protocol, prompts, normalization rules, scorer, code revision, tool versions, input and evidence manifests, environment image, random seeds, session state and every request/result binding. Supply-chain systems similarly distinguish an artifact from a signed statement about the steps and principals that produced it [9, 12]. Our threat model applies this distinction to research execution while declining to infer scientific validity from software integrity.

2.2 Adversary capabilities

The adversary may:

T1. replace an executor or model while copying a declared identity;

T2. replay a valid result under a different request, epoch or evidence set;

T3. let one nominal lane read the other lane's prompt, state or output;

T4. submit the same model or service twice under different lane names;

T5. write a success marker before all output bytes are durable;

T6. provide a non-empty but unauthenticated certificate identifier;

T7. convert a host, network, quota or tool failure into a scientific datum;

T8. omit inconvenient failures or downgrade them to missing rows;

T9. change normalization, scoring or stopping after observing an outcome;

T10. exploit a shared helper, gold function or evidence cache to manufacture agreement.

The adversary need not break a hash. Correct hashes can bind the wrong semantics, and signed statements can be authentic statements from an untrusted or inappropriately scoped issuer. Identity-oriented signing and transparency logs illustrate the additional trust structure required beyond a digest [8].

2.3 Out-of-scope threats

The contract does not decide whether a scientific source is correct, whether a model response is well reasoned, whether an evaluator's ontology is complete, or whether a statistical estimand answers the substantive question. It also does not protect a fully compromised trust root or guarantee confidentiality. These are separate evidence-quality, scientific-design and security questions. A deployment may attach additional controls, but must not advertise them as properties of the base receipt contract.

3 Request and receipt semantics

3.1 Canonical request

Definition 1 (Execution request)**.** *An execution request is the canonical tuple*

$$q = (p, c, i, e, v, t, m, s, b),$$

where p is the protocol digest, c the code digest, i the ordinary input manifest, e the scientific-evidence manifest, v the environment and tool manifest, t the requested terminal schema, m the required executor/model identity policy, s the seed and session policy, and b the resource and access budget. Its request identifier is $H(\mathrm{canon}(q))$.

Canonicalization is part of the protocol, not an implementation detail. Every field that changes observable execution or admissibility must either occur in q or be prohibited. The request identifier therefore binds declared semantics, not only a prompt string. The scientific-evidence manifest is distinct from ordinary inputs because source bytes, credentials, caches and auxiliary configuration have different authority.

3.2 Result receipt

Definition 2 (Result receipt)**.** *A result receipt is*

$$r = (h_q, h_o, z, u, k, g, w, \tau, \sigma),$$

where h_q is the request identifier, h_o the output-manifest digest, z a typed terminal, u an authenticated executor subject, k provider/model/checkpoint identity when applicable, g the realized code/tool/environment configuration, w the evidence-lineage graph, τ timing and epoch data, and σ an attestation over the canonical receipt payload.

The terminal z belongs to a closed sum type:

$$z \in \{\textsc{completed}, \textsc{valid-negative}, \textsc{cannot-check}, \textsc{execution-failure}, \textsc{invalid-result}\}.$$

A host or capability failure is therefore not encoded as a numerical outcome or a successful empty result. A valid negative is a result of the scientific protocol; an execution failure is a result of the computing environment. Conflating them is a type error.

3.3 Verification predicates

For a request q, receipt r and trust policy T, define:

$$\begin{aligned}
S(q, r) &: \text{ schema and canonicalization are valid,}\\
D(q, r) &: \text{ request, output and manifest digests match,}\\
I(q, r, T) &: \text{ executor/model identities satisfy } T,\\
C(q, r, T) &: \text{ attestation chain and issuer scope satisfy } T,\\
E(q, r) &: \text{ evidence lineage is complete for the claimed level,}\\
Z(r) &: \text{ terminal is admissible for the requested scientific endpoint.}
\end{aligned}$$

The mechanical admission predicate is

$$M(q, r, T) = S \wedge D \wedge I \wedge C \wedge E.$$

Scientific admission additionally requires a frozen adjudication rule F:

$$\mathrm{Admit}(q, r, T, F) = M(q, r, T) \wedge Z(r) \wedge F(q, r).$$

No missing predicate is coerced to true. Missing or unverifiable identity, lineage, trust or protocol data produce CANNOT-CHECK for the associated claim.

4 Attribution and certificate trust

Content addressing and attribution answer different questions. A digest can show that two observers refer to the same bytes, but it does not show who generated those bytes, under which authority, or whether the declared execution occurred. Conversely, an authenticated principal can sign a misleading or incomplete statement. The receipt must bind both content and a scoped attestation.

Definition 3 (Attribution-complete receipt)**.** *A receipt is attribution-complete for an execution claim only if its attested payload binds the executor principal, issuer and subject namespace, host/runtime, provider/model/checkpoint when used, inference or execution configuration, protocol/prompt digest, tool versions, code revision, environment digest, evidence-manifest digest, seed, session, campaign, request and result digests.*

This definition is deliberately stronger than requiring a non-empty executor, model or certificate string. Attestation systems such as in-toto bind supply-chain steps to authorized actors [12]; SLSA provenance records the build definition and producing platform [9]; and identity-based signing can connect a short-lived certificate, an identity and a transparency log entry [8]. These are relevant design donors, not evidence that a research receipt inherits their guarantees.

Definition 4 (Certificate trust predicate)**.** *For certificate c and policy T,* $\mathrm{Trust}(c, T)$ *holds only when the signature verifies to a configured root, the issuer and subject namespaces are allowed, the subject is authorized for the requested role, the validity epoch covers the execution, revocation and transparency requirements pass, and the signed payload binds the receipt's canonical bytes.*

An arbitrary identifier fails this predicate even if it is unique. A certificate for code signing also cannot automatically authorize scientific adjudication; role and scope are part of the policy. When no appropriate trust root is configured, the system may preserve the self-declared metadata for audit but must return CANNOT-CHECK for authenticated attribution.

Environment attribution is similarly substantive. Tools such as ReproZip capture files, libraries and configuration needed to rerun a computational experiment [3], while workflow packages can bundle plans, runs, inputs and outputs [4, 5]. A receipt contract should interoperate with these representations rather than replace them. Its added requirement is that the environment record be content-bound to the specific request and result and assessed against the claim being made.

5 Separating execution failure from scientific evidence

Let R_q be all structurally valid receipts for request q. Partition them into completed outcomes R_q^+, valid scientific negatives R_q^-, uncheckable attempts $R_q^?$ and execution failures R_q^f. Every attempt should be retained in an execution ledger, because omission can hide reliability failures. Only the first two sets are candidates for a scientific evidence ledger, and both remain subject to the frozen adjudication F.

Proposition 1 (Failure non-contamination)**.** *If $Z(r)$ accepts only terminals in $R_q^+ \cup R_q^-$, then no receipt in R_q^f can enter the admitted scientific-evidence set through* Admit*.*

Proof. For $r \in R_q^f$, the closed terminal type assigns $z =$ EXECUTION-FAILURE. By definition, $Z(r)$ is false for this terminal. Therefore the conjunction defining Admit is false independently of all other predicates. □

This is a contract theorem, not empirical evidence that an implementation always enforces the contract. A hostile implementation can misclassify a network failure as a completed empty result, bypass the typed constructor or mutate stored bytes. The adversarial protocol in Section 7 must therefore test the implementation boundary.

Proposition 2 (No silent row deletion)**.** *If every accepted request identifier must terminate in exactly one sealed receipt or one explicit missing-receipt alarm, then a campaign summary cannot convert an execution failure into an absent observation without violating the coverage predicate.*

Proof. The coverage predicate compares the frozen request set with the disjoint union of sealed receipts and explicit alarms. Removing a failed attempt from both sets makes the union smaller than the request set, so coverage fails. □

Valid negative results require the opposite treatment: they must remain in the scientific ledger. Preregistration and Registered Reports address outcome-dependent analysis and publication incentives [2, 7]. The execution contract supports that separation by binding the protocol and terminal schema before outcome access, but cannot determine whether the registered scientific design is meaningful.

Repair verification is also distinct from scientific replication. Rerunning a fixed case after correcting a digest, parser or environment defect can establish that the repaired implementation meets the same contract. Unless the scientific protocol prospectively defines the repair run as a new independent unit, it cannot be counted as additional corroboration.

6 An independence taxonomy

The word *independent* is often applied to any two labeled runs. We replace that binary label with six ordered claim levels. Each level names the strongest separation that has actually been verified.

Definition 5 (Agreement levels)**.** *For a pair or set of executions, define:*

- ***L1.*** Deterministic structural agreement*: no model calls; the same frozen inputs are checked by deterministic procedures.*
- ***L2.*** Cross-implementation agreement*: different decision code paths agree, but evidence, executor or model may be shared.*
- ***L3.*** Executor-independent replication*: authenticated executor principals, sessions and workspaces are disjoint.*
- ***L4.*** Model-independent replication*: provider and immutable checkpoint identities are disjoint under a frozen admissibility rule.*
- ***L5.*** Evidence-independent corroboration*: upstream evidence acquisition and lineage are disjoint before adjudication.*
- ***L6.*** Scientific corroboration*: evidence, execution and adjudication are independent under a preregistered multi-instance scientific design.*

The levels are an authority ordering, not a claim that every application needs the highest level. A deterministic source audit may be valuable at L1. It becomes misleading only when described as L3–L6.

For each level L_j, let G_j be the set of required verified gates. Define the reported level

$$\ell^* = \max\{j : \forall g \in G_j,\ g = \text{pass}\}.$$

Unknown is not pass. If no gate set is complete, the result is reported as unclassified agreement with explicit failed or uncheckable predicates.

Proposition 3 (Monotone downgrade)**.** *If a required gate for level L_j is false or uncheckable, no label L_k with $k \geq j$ is admissible.*

Proof. The gate sets are cumulative: $G_j \subseteq G_k$ for $k \geq j$. Therefore a failed or unknown member of G_j is also an unsatisfied member of G_k, so neither level satisfies the definition of ℓ^*. □

6.1 Required gates

The cumulative gates are:

G1. protocol, prompts, normalization, scorer, endpoints and intended level are frozen before either output;

G2. executor principals are authenticated and distinct when executor independence is claimed;

G3. provider and checkpoint identities are distinct when model independence is claimed;

G4. sessions, workspaces, stores, seeds and temporary state are disjoint;

G5. neither lane can read the other's prompt, state, receipt or output before both terminals are sealed;

G6. implementations share no hidden gold function or copied decision result, and common libraries are enumerated;

G7. evidence-lineage graphs are disjoint when evidence independence is claimed;

G8. scoring is performed by a frozen third component or blinded adjudicator;

G9. all attribution fields required by the level are bound in each receipt;

G10. certificate issuer, subject, scope and signature satisfy the trust policy;

G11. the substitution, replay, leakage, race and forgery attack suite passes;

G12. general reliability uses prospectively sampled task or campaign units and task-level uncertainty rather than repeated traces from one question.

Lane names, process identifiers and different prompt strings are not substitutes for these gates. In particular, two logical lanes routed through one broker and one model are at most cross-implementation agreement if their decision code paths are actually distinct; otherwise they are repeated requests.

7 Prospective adversarial protocol

The framework is falsifiable only if every claimed guarantee has an attack that can make it fail. The protocol below is therefore part of the scientific object, not an optional engineering appendix. Exact fixtures, acceptance predicates, normalization and stopping rules must be frozen before protected execution.

7.1 Registered attacks

A1. Executor substitution. Replace the authenticated principal while retaining the declared executor string. Attribution must fail.

A2. Model substitution. Change provider or checkpoint under the same lane label. Model-independence classification must fail or downgrade.

A3. Evidence replay. Reuse a receipt under a different evidence manifest or epoch. Request binding must fail.

A4. Cross-lane leakage. Expose one lane's intermediate output before the other seals. The no-cross-read gate must fail.

A5. Shared-decision helper. Route nominally separate implementations through one gold function. The implementation gate must fail.

A6. Certificate forgery. Supply a non-empty identifier, wrong issuer, wrong subject or correctly signed but out-of-scope certificate. Trust must fail closed.

A7. Stale result. Replay a valid receipt after protocol, code, tool, environment or campaign epoch changes. Digest or epoch verification must fail.

A8. Partial write. Interrupt publication between payload and terminal writes. No completed receipt may become visible.

A9. Invalid success. Return malformed or incomplete content with a success transport status. Schema or terminal validation must fail.

A10. Capability failure. Inject network, quota, process and tool errors. They must enter the execution ledger and remain outside scientific evidence.

A11. Race. Concurrent attempts target the same receipt identifier. Publication must be atomic or explicitly detect conflict.

A12. Selective omission. Remove a failed attempt before summarization. Coverage verification must fail.

7.2 Units and endpoints

The unit of a conformance test is a frozen attack fixture. Passing all fixtures supports only the statement that the evaluated implementation met those cases. A reliability study requires independently sampled campaigns or tasks, a declared failure taxonomy, repeated but non-pseudoreplicated units, uncertainty at the campaign/task level and a sampling frame that includes realistic infrastructure and semantic failures.

Primary conformance endpoints are exact: false admission count, missed failure count, incorrect independence label count, stale/replayed receipt acceptance count, and atomic-publication violations. Runtime and storage cost are secondary. Every attack family must contain a positive fixture and a negative fixture so that a checker returning constant failure or constant success cannot pass.

7.3 Custody and analysis

Attack generation, implementation under test and adjudication should be separated. The frozen scorer consumes only sealed receipts and attack truth unavailable to the implementation. Development fixtures and protected fixtures must have disjoint identifiers and manifests. Any post-outcome repair starts a new conformance version and preserves the failed version in the evidence history.

Computational-reproducibility guidance emphasizes accessible data, code and workflows [11], while artifact review distinguishes availability, functional quality and reproduced or replicated results [1]. The proposed protocol similarly refuses to infer a stronger claim merely because an artifact runs.

8 Results status

No protected paper result exists. Unit tests and deterministic scans establish only conformance of a checked implementation to their registered cases. They do not estimate general reliability, authenticate all executor claims, or establish scientific corroboration.

This chapter is intentionally not populated with prospective tables or inferred numbers. A publication result may be added only after the protocol in Section 7 is frozen on an immutable digest and executed by an identified implementation under the declared custody plan.

When results exist, the primary table must contain one row per attack family and report the frozen fixture count, false admissions, false rejections, coverage alarms, incorrect labels, execution environment digest, receipt-set digest and adjudication receipt. A separate reliability table may be reported only if the multi-instance sampling gate passes. The abstract, title and conclusion must name the highest verified independence level and must report all failed or CANNOT-CHECK gates.

Existing engineering checks, if cited in a future version, belong in a development history table. They may motivate attacks or demonstrate that the protocol is executable, but they cannot be promoted retrospectively into protected paper evidence. Likewise, agreement between deterministic auditors on one source tree is an

L1 structural result; zero model calls remove model-generated semantic judgment from that audit but do not create evidence or scientific independence.

The absence of a result is an explicit status and must not be hidden. It is the present status of a framework paper whose nontrivial empirical claims are prospective. This separation prevents the manuscript structure from manufacturing an apparent positive terminal before the evidence exists.

9 Limitations and authority boundary

The formal properties in this paper are consequences of an explicit contract. They do not show that an implementation is free from bypasses, that a trust root is secure, or that the declared environment captures every causal dependency. The adversarial protocol can increase confidence only within its attack model and sampling frame.

The independence ordering is deliberately conservative but not universally total. Different applications may value model, executor and evidence independence differently. For example, two independently implemented deterministic checkers over the same evidence may provide a stronger software-bug diagnostic than two models from different providers that share copied evidence and scoring code. The framework therefore requires the component predicates to be reported alongside the level.

Cryptographic attribution introduces operational dependencies: issuer governance, credential security, revocation, transparency infrastructure and privacy. Some research settings cannot reveal executor or evidence identities publicly. A future extension should support privacy-preserving attestations while retaining enough verifiability to prevent self-declared identity from satisfying an independence claim.

Complete environment capture is also difficult. A container digest does not bind an external service's mutable behavior, a model alias may not identify an immutable checkpoint, and hardware or nondeterministic libraries can change outputs. Receipt schemas must allow CANNOT-CHECK rather than encode unavailable metadata as empty but acceptable fields.

Most importantly, provenance is not epistemic warrant. FAIR principles concern the findability, accessibility, interoperability and reuse of digital objects [13]; provenance standards describe how objects were produced [6]. Neither determines whether a premise is true, a measurement is valid, a comparator is fair or a conclusion follows. Scientific authority remains with the study design, evidence and adjudication, not the execution layer.

The next step is a preregistered hostile conformance study conducted under independent custody. A later multi-campaign study would be needed for any general reliability estimate. Until those gates close, the contribution is the framework, formal separation and falsifiable protocol.

10 Reproducibility and relation to prior work

10.1 Provenance and workflow packaging

W3C PROV supplies a domain-independent model of entities, activities and agents [6]. CWLProv maps workflow executions into interoperable provenance records [4]; RO-Crate packages research artifacts and contextual metadata [10]; and Workflow Run RO-Crate extends that packaging to multiple granularities of execution provenance [5]. This paper does not replace those models. It isolates a narrower boundary: which additional trust, terminal and independence predicates are required before execution records support specific research claims.

10.2 Attestation and supply-chain integrity

in-toto protects a software supply chain by linking authorized steps and materials [12]. SLSA defines provenance for artifacts produced by a build platform [9], and Sigstore combines identity-based signing with transparency infrastructure [8]. These systems motivate authenticated, scoped receipts and substitution attacks. The distinction is that a scientifically admissible result needs more than an authentic execution: it also needs a frozen evidence and adjudication protocol.

10.3 Computational reproducibility and prospective control

ReproZip captures dependencies and environments for computational experiments [3]; broader reproducibility guidance calls for available and cited data, code and workflows [11]; and ACM artifact badging separates artifact properties from reproduced or replicated results [1]. Preregistration and Registered Reports address analytic and publication decisions made after outcomes are known [2, 7]. The present framework connects these lines through a fail-closed rule: the receipt may support only the claim whose protocol, identities and independence predicates it actually binds.

10.4 Artifact requirements for this paper

A submission package must contain complete tracked TeX chapters, the bibliography, the canonical build command and dependency versions, a source-closure manifest, the protocol and claim-ledger digests, the attack fixtures after protected release, the raw receipt set, verification and adjudication code, and a source-to-PDF text equivalence report. A clean build must not depend on an untracked cache or runtime network access.

The current manuscript is a framework version. It contains no protected receipt set and therefore makes no implementation-performance or reliability claim. This status must remain synchronized across the abstract, Results chapter, claim ledger, README and generated PDF.

11 Conclusion

Research-execution records are valuable only when their authority is stated precisely. This paper separates request identity, result integrity, executor attribution, evidence admission and scientific authority; defines a six-level independence taxonomy; and specifies hostile tests that can falsify each guarantee. The formal contract prevents typed capability failures from entering scientific evidence and prevents an agreement label from exceeding its weakest verified gate. These are framework properties, not a reliability result. The immediate empirical task is therefore not to claim successful independent research execution, but to freeze and run the adversarial protocol under authenticated, separated custody.

References

[1] Association for Computing Machinery. Artifact review and badging, version 1.1. Publication policy, 2020. URL https://www.acm.org/publications/policies/artifact-review-and-badging-current.

[2] Christopher D. Chambers, Zoltan Dienes, Robert D. McIntosh, Pia Rotshtein, and Klaus Willmes. Registered reports: Realigning incentives in scientific publishing. *Cortex*, 66:A1–A2, 2015. doi: 10.1016/j.cortex.2015.03.022.

[3] Fernando Chirigati, Rémi Rampin, Dennis Shasha, and Juliana Freire. ReproZip: Computational reproducibility with ease. In *Proceedings of the 2016 International Conference on Management of Data*, pages 2085–2088. ACM, 2016. doi: 10.1145/2882903.2899401.

[4] Farah Zaib Khan, Stian Soiland-Reyes, Richard O. Sinnott, Andrew Lonie, Carole Goble, and Michael R. Crusoe. Sharing interoperable workflow provenance: A review of best practices and their practical application in CWLProv. *GigaScience*, 8(11):giz095, 2019. doi: 10.1093/gigascience/giz095.

[5] Simone Leo, Michael R. Crusoe, Laura Rodríguez-Navas, Raül Sirvent, Alexander Kanitz, Paul De Geest, Rudolf Wittner, Luca Pireddu, Daniel Garijo, José M. Fernández, et al. Recording provenance of workflow runs with RO-Crate. *PLOS ONE*, 19(9):e0309210, 2024. doi: 10.1371/journal.pone.0309210.

[6] Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, April 2013. URL https://www.w3.org/TR/prov-dm/.

[7] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

[8] Sigstore Community. Signing overview. Technical documentation, 2026. URL https://docs.sigstore.dev/cosign/signing/overview/. Accessed 22 August 2026.

[9] SLSA Community. SLSA build provenance, specification v1.2. Technical specification, 2026. URL https://slsa.dev/spec/v1.2/build-provenance. Accessed 22 August 2026.

[10] Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, José M. Fernández, Daniel Garijo, Björn Grüning, Marco La Rosa, Simone Leo, Eoghan Ó Carragáin, Marc Portier, Ana Trisovic, RO-Crate Community, Paul Groth, and Carole Goble. Packaging research artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022. doi: 10.3233/DS-210053.

[11] Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. Enhancing reproducibility for computational methods. *Science*, 354(6317):1240–1241, 2016. doi: 10.1126/science.aah6168.

[12] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410. USENIX Association, 2019. URL https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

[13] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18.